

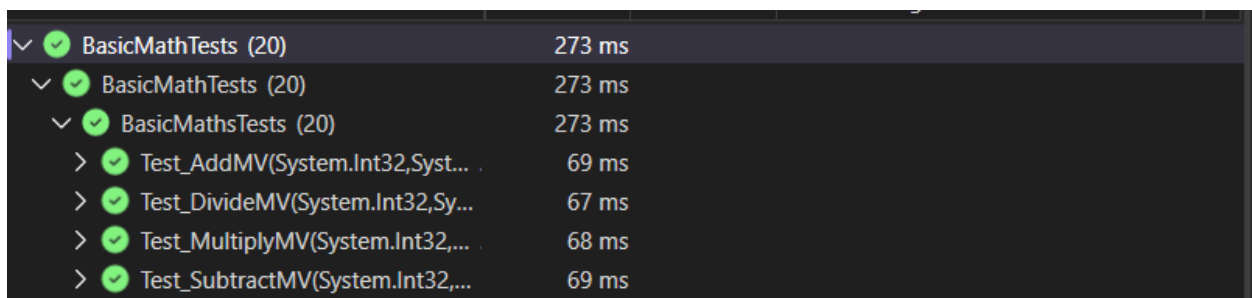
Introduction

In this lab, I focused on applying fundamental software testing techniques to validate application logic and automate browser interactions . The primary purpose of these exercises was to gain hands-on experience with Unit Testing to verify individual code components in isolation and Automation Testing to execute repeatable, script-driven tests on a web application .

Exercises

Exercise 1

I created a C# .NET Core Console application containing a BasicMaths class and a corresponding Unit Test project . I wrote isolated tests for addition, subtraction, multiplication, and division methods . To ensure thorough coverage, I applied Equivalence Partitioning (EP) to test positive, negative, and zero values , as well as Boundary Value Analysis (BVA) to test the upper and lower limits of the integer data type .



A screenshot of a test runner interface, likely Test Explorer in Visual Studio, showing a tree view of test results. The root node is 'BasicMathTests (20)' with a green checkmark and a duration of 273 ms. It is expanded to show a sub-node 'BasicMathTests (20)' (also with a green checkmark and 273 ms), which is further expanded to show four individual test methods, each with a green checkmark and a duration: 'Test_AddMV(System.Int32,Syst...' (69 ms), 'Test_DivideMV(System.Int32,Sy...' (67 ms), 'Test_MultiplyMV(System.Int32,...' (68 ms), and 'Test_SubtractMV(System.Int32,...' (69 ms).

✓ BasicMathTests (20)	273 ms
✓ BasicMathTests (20)	273 ms
✓ BasicMathsTests (20)	273 ms
> ✓ Test_AddMV(System.Int32,Syst...	69 ms
> ✓ Test_DivideMV(System.Int32,Sy...	67 ms
> ✓ Test_MultiplyMV(System.Int32,...	68 ms
> ✓ Test_SubtractMV(System.Int32,...	69 ms

Exercise 2

To make the unit tests more scalable, I implemented data-driven testing by externalizing the test inputs and expected outputs . Using the CsvHelper library, I configured the test methods to read multiple test scenarios directly from external CSV files, such as add_testdata.csv and divide_testdata.csv .

▼	✖ BasicMathXTests (16)	20 ms	
▼	✖ BasicMathXTests (16)	20 ms	
▼	✖ CalculatorCsvTests (16)	20 ms	
▼	✖ Add_CsvData_ReturnsCorrectS...	20 ms	
	✔ Add_CsvData_ReturnsCorrect...	< 1 ms	
	✔ Add_CsvData_ReturnsCorrect...	< 1 ms	
	✔ Add_CsvData_ReturnsCorrect...	17 ms	
	✖ Add_CsvData_ReturnsCorrect...	3 ms	Assert.Equal() Failure: Values differ Exp...
>	✔ Divide_CsvData_ReturnsCorrect...	< 1 ms	
>	✔ Multiply_CsvData_ReturnsCorre...	< 1 ms	
▼	✖ Subtract_CsvData_ReturnsCorre...	< 1 ms	
	✔ Subtract_CsvData_ReturnsCor...	< 1 ms	
	✖ Subtract_CsvData_ReturnsCor...	< 1 ms	System.ArgumentException : Object of...
	✔ Subtract_CsvData_ReturnsCor...	< 1 ms	
	✔ Subtract_CsvData_ReturnsCor...	< 1 ms	

Exercise 3

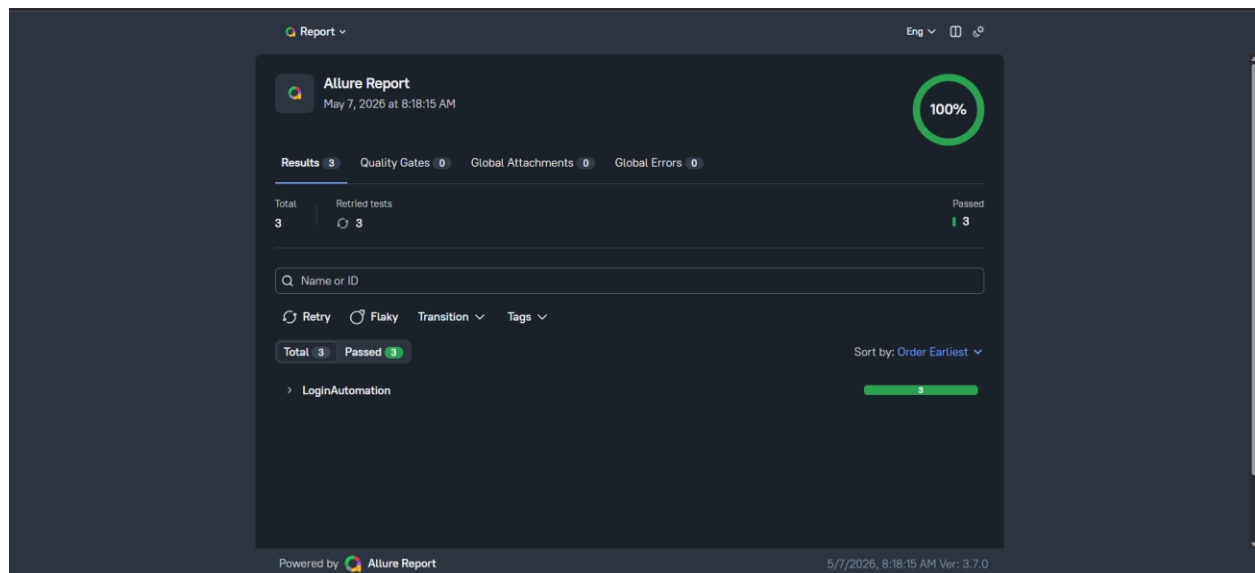
I applied the unit testing concepts to a practical BankAccount scenario, testing the Debit, Credit, and Withdraw methods . I utilized a BankAccountTestData.csv file to simulate multiple user transactions . This exercise was particularly useful for learning how to test for expected errors, such as asserting that an InvalidOperationException is correctly thrown when a transaction results in "Insufficient funds" .

Test	Duration	Traits	Error Message
✓ BankAccountTests (14)	903 ms		
✓ BankAccountTests (14)	903 ms		
✓ BankAccountCsvTests (4)	452 ms		
✓ Debit_CsvData_UpdatesBalance ..	452 ms		
✓ Debit_CsvData_UpdatesBalan...	451 ms		
✓ Debit_CsvData_UpdatesBalan...	1 ms		
✓ Debit_CsvData_UpdatesBalan...	< 1 ms		
✓ Debit_CsvData_UpdatesBalan...	< 1 ms		
✓ BankAccountTests (10)	451 ms		
✓ Credit_ValidAmount_UpdatesB...	< 1 ms		
✓ Credit_ValidAmount_Updates...	< 1 ms		
✓ Credit_ValidAmount_Updates...	< 1 ms		
✓ Credit_ValidAmount_Updates...	< 1 ms		
✓ Debit_InsufficientFunds_Throws...	< 1 ms		
✓ Debit_ValidAmount_UpdatesBa...	451 ms		
✓ Debit_ValidAmount_UpdatesB...	451 ms		
✓ Debit_ValidAmount_UpdatesB...	< 1 ms		
✓ Debit_ValidAmount_UpdatesB...	< 1 ms		
✓ Withdraw_ValidAmount_Updat...	< 1 ms		
✓ Withdraw_ValidAmount_Upda...	< 1 ms		
✓ Withdraw_ValidAmount_Upda...	< 1 ms		
✓ Withdraw_ValidAmount_Upda...	< 1 ms		

Exercise 4

Transitioning to UI automation, I set up a project using Selenium WebDriver and C# . After creating a simple HTML login form , I wrote an automated script that navigated to the page, located the username and password fields by their HTML IDs, and simulated a user clicking the login button . I also integrated a LoginTestData.csv file to automate the login process across multiple different user credentials and reviewed the steps for generating detailed execution reports using Allure .

Test	Duration	Traits	Error Message
✓ LoginAutomation (3)	12.3 sec		
✓ LoginAutomation (3)	12.3 sec		
✓ LoginTests (3)	12.3 sec		
> ✓ Login_WithValidCredentials_Sh...	12.3 sec		



Conclusion

Overall, this lab provided a highly practical learning experience in establishing a robust testing environment within Visual Studio . I learned how to isolate backend logic using unit tests and external data sources , and how to programmatically drive browser interactions to replace manual UI testing . One minor challenge I encountered was correctly configuring the absolute file paths to ensure the test runner could locate the external CSV files and the local HTML login page . Troubleshooting these configuration steps ultimately deepened my understanding of how automated testing frameworks interact with local file systems.

Appendix

Source Code Repository:

- [Exercise 1+2](#)
- [Exercise 3](#)
- [Exercise 4](#)